

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Consultas sobre la base de ventas

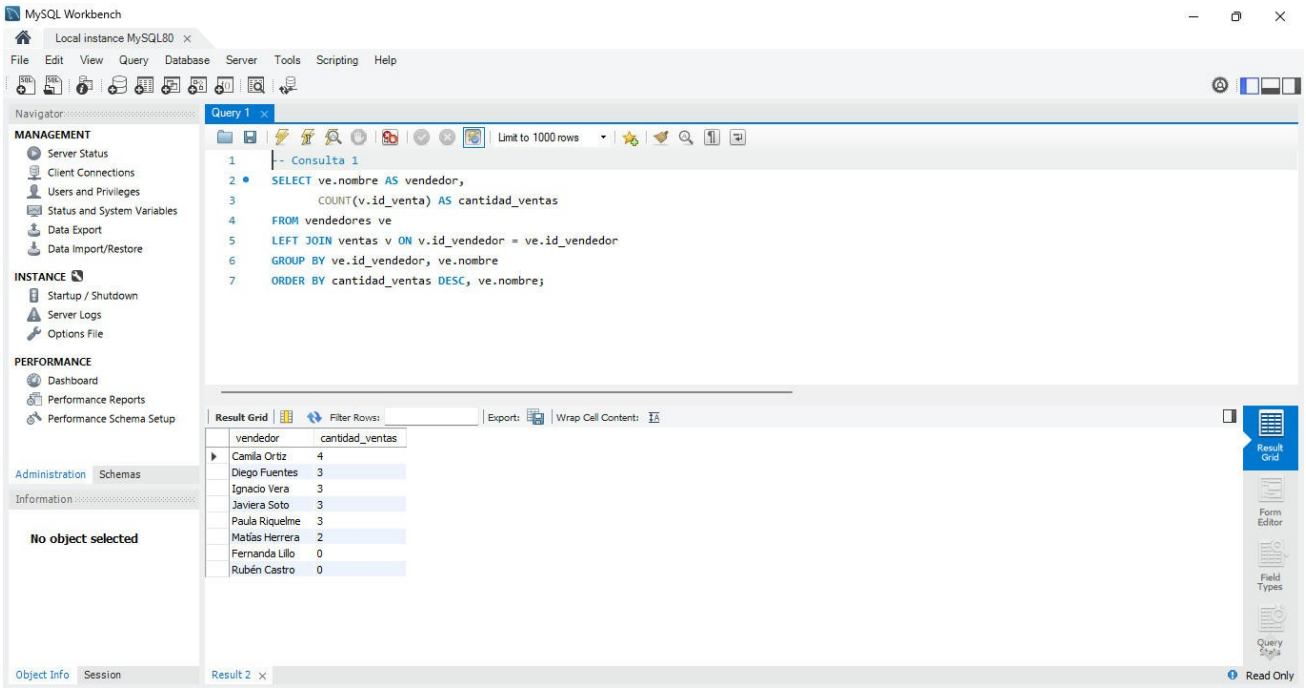
Creación de la base, carga oficial y consultas en MySQL

Andes Bike · Laboratorio 5

Asignatura	ICN-292 Sistemas de Información para la Gestión
Nombre	Daniela López
Rol USM	2023602-6
Paralelo	101
Fecha	13-11-2026
Repositorio	https://github.com/Danielalopezg/ICN292-Lab5-Lopez-Daniela

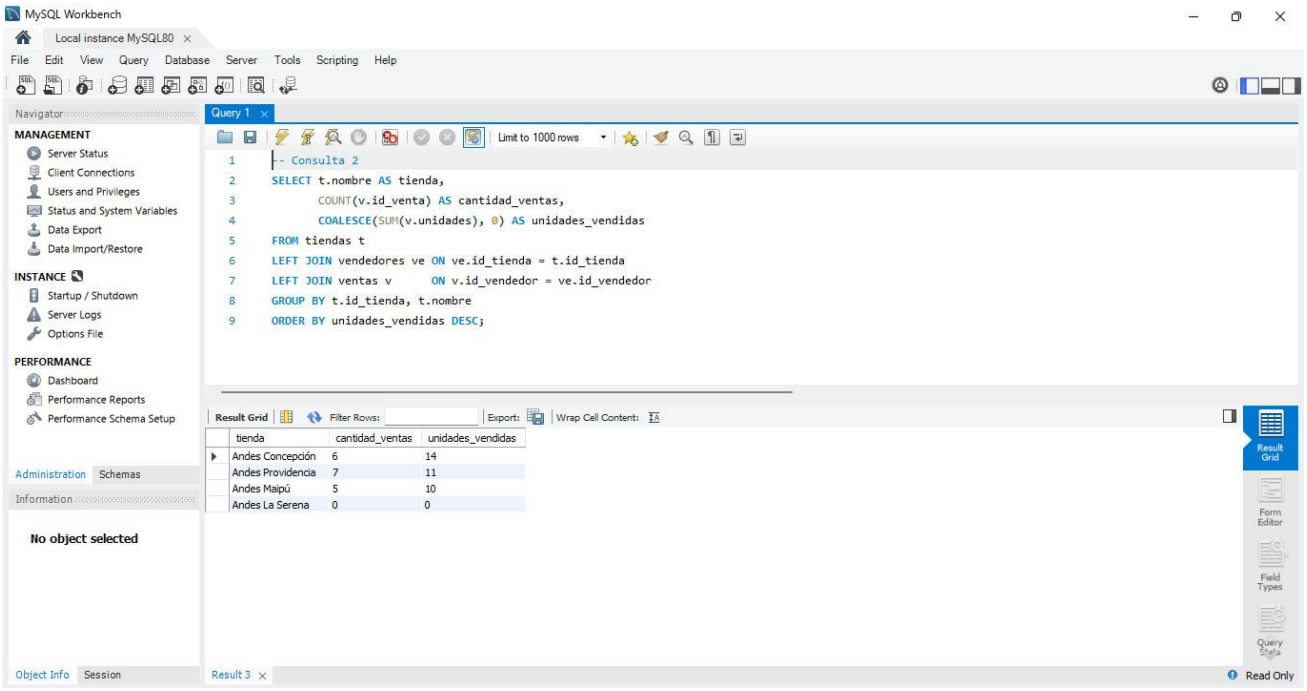
Consulta 1

```
SELECT ve.nombre AS vendedor,
       COUNT(v.id_venta) AS cantidad_ventas
FROM vendedores ve
LEFT JOIN ventas v ON v.id_vendedor = ve.id_vendedor
GROUP BY ve.id_vendedor, ve.nombre
ORDER BY cantidad_ventas DESC, ve.nombre;
```



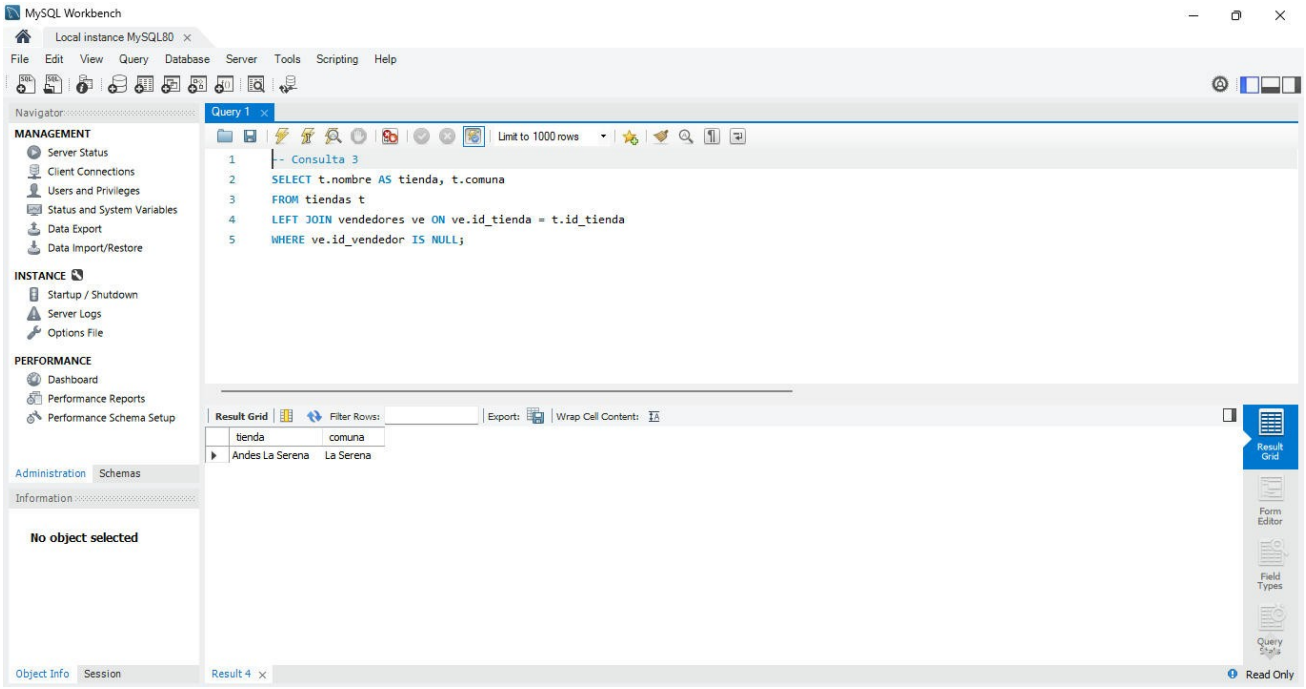
Consulta 2

```
SELECT t.nombre AS tienda,
       COUNT(v.id_venta) AS cantidad_ventas,
       COALESCE(SUM(v.unidades), 0) AS unidades_vendidas
FROM tiendas t
LEFT JOIN vendedores ve ON ve.id_tienda = t.id_tienda
LEFT JOIN ventas v      ON v.id_vendedor = ve.id_vendedor
GROUP BY t.id_tienda, t.nombre
ORDER BY unidades_vendidas DESC;
```



Consulta 3

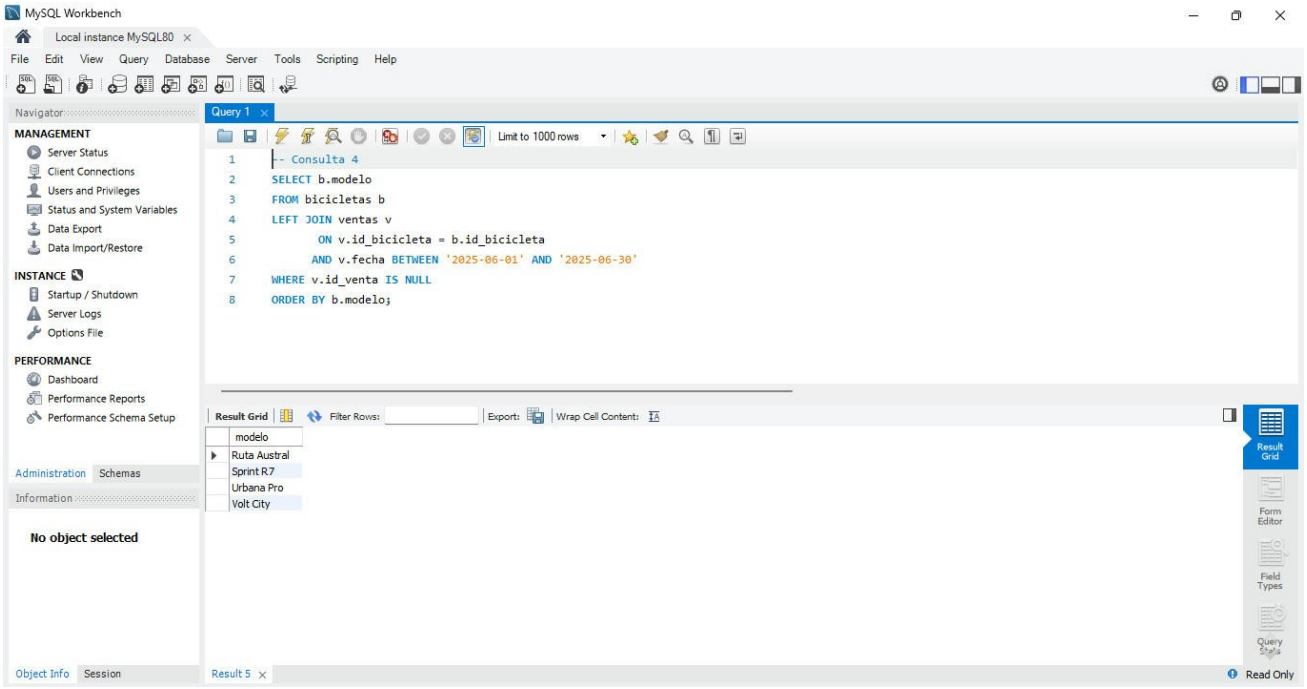
```
SELECT t.nombre AS tienda, t.comuna
FROM tiendas t
LEFT JOIN vendedores ve ON ve.id_tienda = t.id_tienda
WHERE ve.id_vendedor IS NULL;
```



Consulta 4

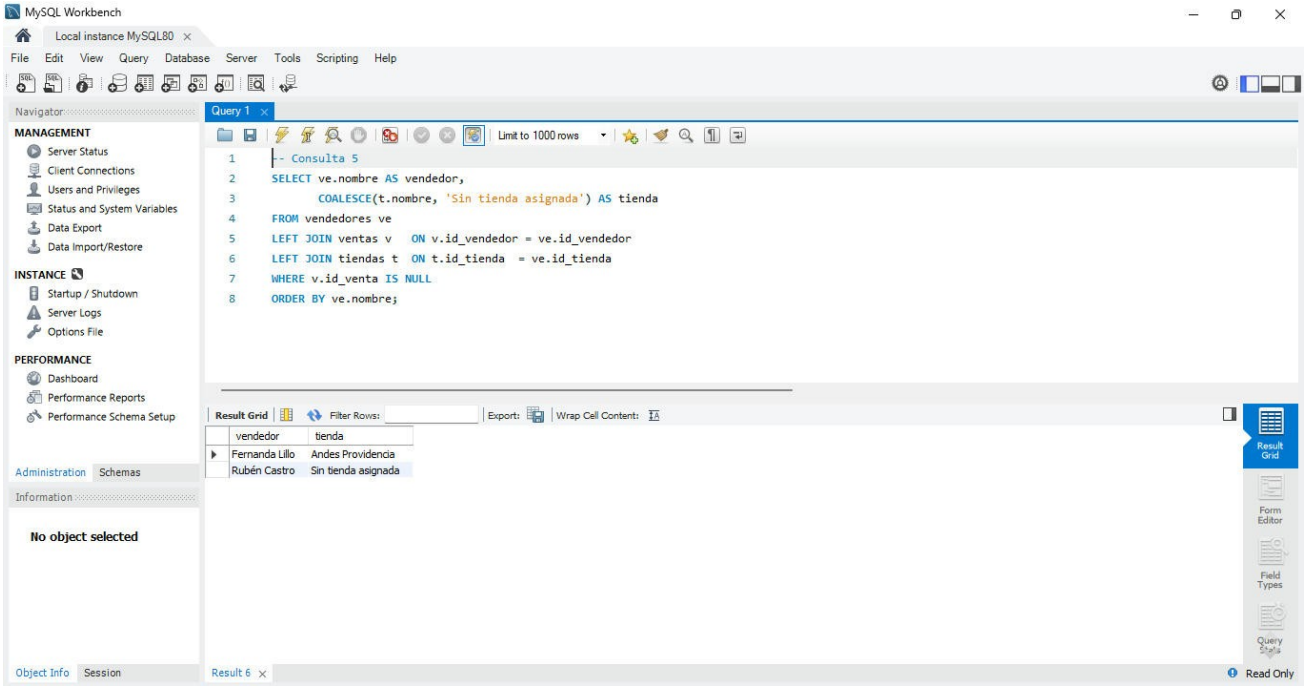
```
SELECT b.modelo
FROM bicicletas b
LEFT JOIN ventas v
       ON v.id_bicicleta = b.id_bicicleta
      AND v.fecha BETWEEN '2025-06-01' AND '2025-06-30'
WHERE v.id_venta IS NULL
ORDER BY b.modelo;
```

La condición de junio de 2025 se puso en el ON del LEFT JOIN, así que solo se cruzan las ventas de ese mes y las bicicletas sin ventas en junio quedan con NULL; si se hubiera puesto en el WHERE, esas filas con NULL no la cumplirían y, junto con v.id_venta IS NULL, la consulta no habría devuelto ninguna fila.



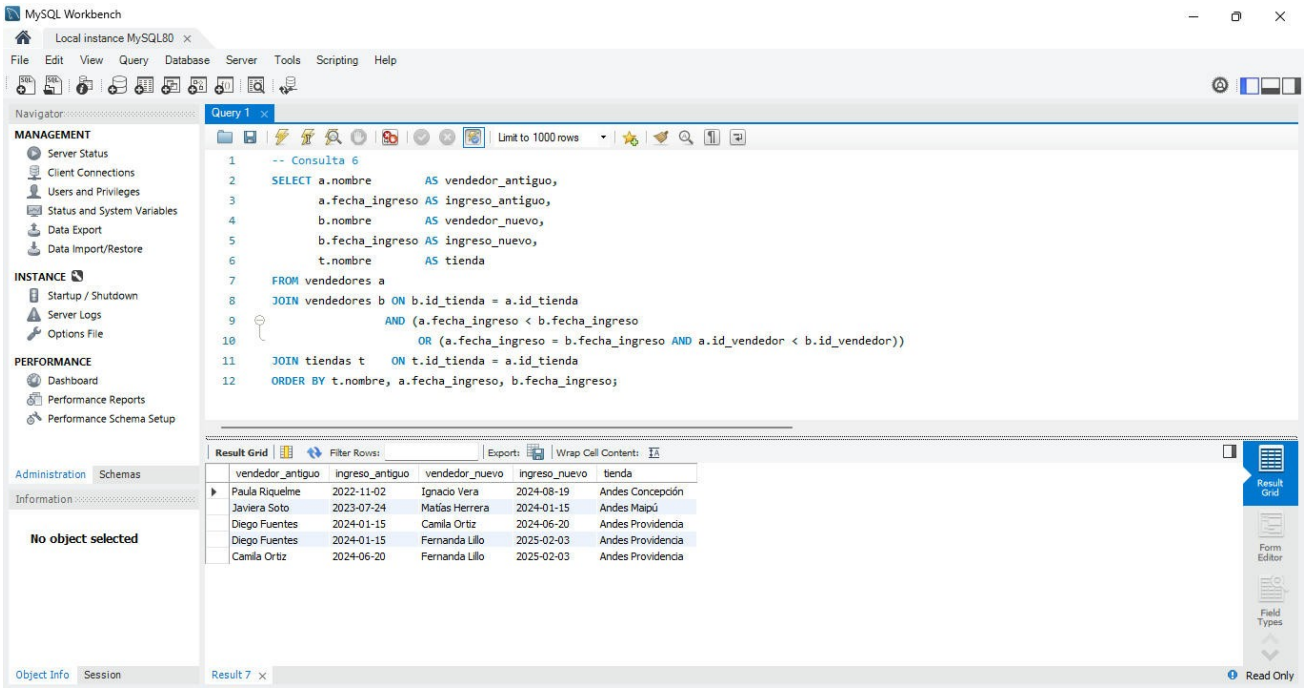
Consulta 5

```
SELECT ve.nombre AS vendedor,
       COALESCE(t.nombre, 'Sin tienda asignada') AS tienda
FROM vendedores ve
LEFT JOIN ventas v   ON v.id_vendedor = ve.id_vendedor
LEFT JOIN tiendas t  ON t.id_tienda  = ve.id_tienda
WHERE v.id_venta IS NULL
ORDER BY ve.nombre;
```



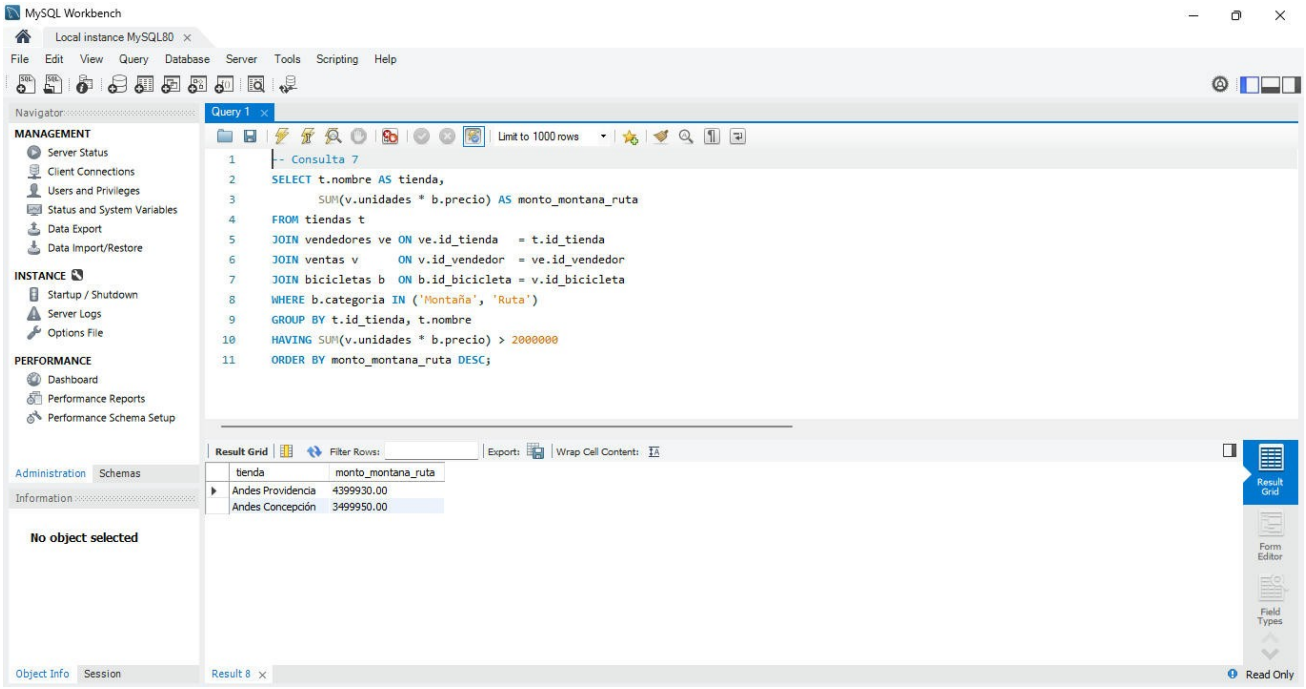
Consulta 6

```
SELECT a.nombre        AS vendedor_antiguo,
       a.fecha_ingreso AS ingreso_antiguo,
       b.nombre        AS vendedor_nuevo,
       b.fecha_ingreso AS ingreso_nuevo,
       t.nombre        AS tienda
FROM vendedores a
JOIN vendedores b ON b.id_tienda = a.id_tienda
                 AND (a.fecha_ingreso < b.fecha_ingreso
                      OR (a.fecha_ingreso = b.fecha_ingreso AND a.id_vendedor <
b.id_vendedor))
JOIN tiendas t    ON t.id_tienda = a.id_tienda
ORDER BY t.nombre, a.fecha_ingreso, b.fecha_ingreso;
```



Consulta 7

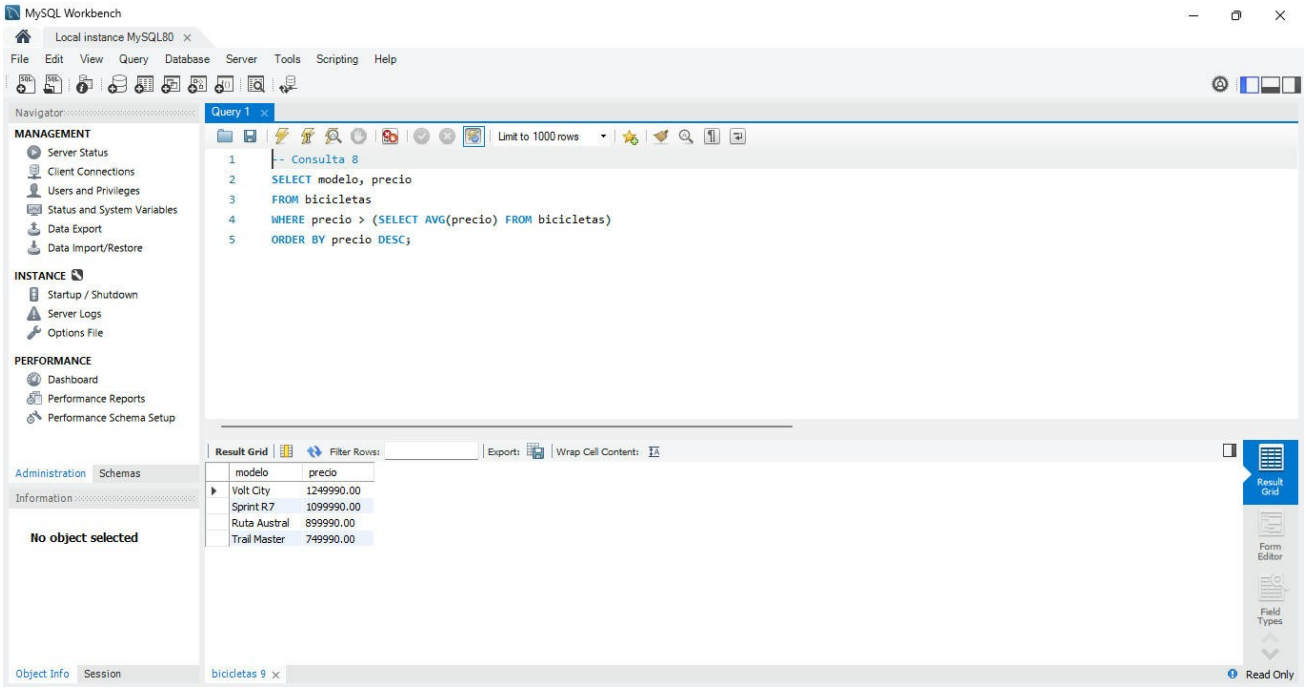
```
SELECT t.nombre AS tienda,
       SUM(v.unidades * b.precio) AS monto_montana_ruta
FROM tiendas t
JOIN vendedores ve ON ve.id_tienda   = t.id_tienda
JOIN ventas v      ON v.id_vendedor  = ve.id_vendedor
JOIN bicicletas b  ON b.id_bicicleta = v.id_bicicleta
WHERE b.categoria IN ('Montaña', 'Ruta')
GROUP BY t.id_tienda, t.nombre
HAVING SUM(v.unidades * b.precio) > 2000000
ORDER BY monto_montana_ruta DESC;
```



Consulta 8

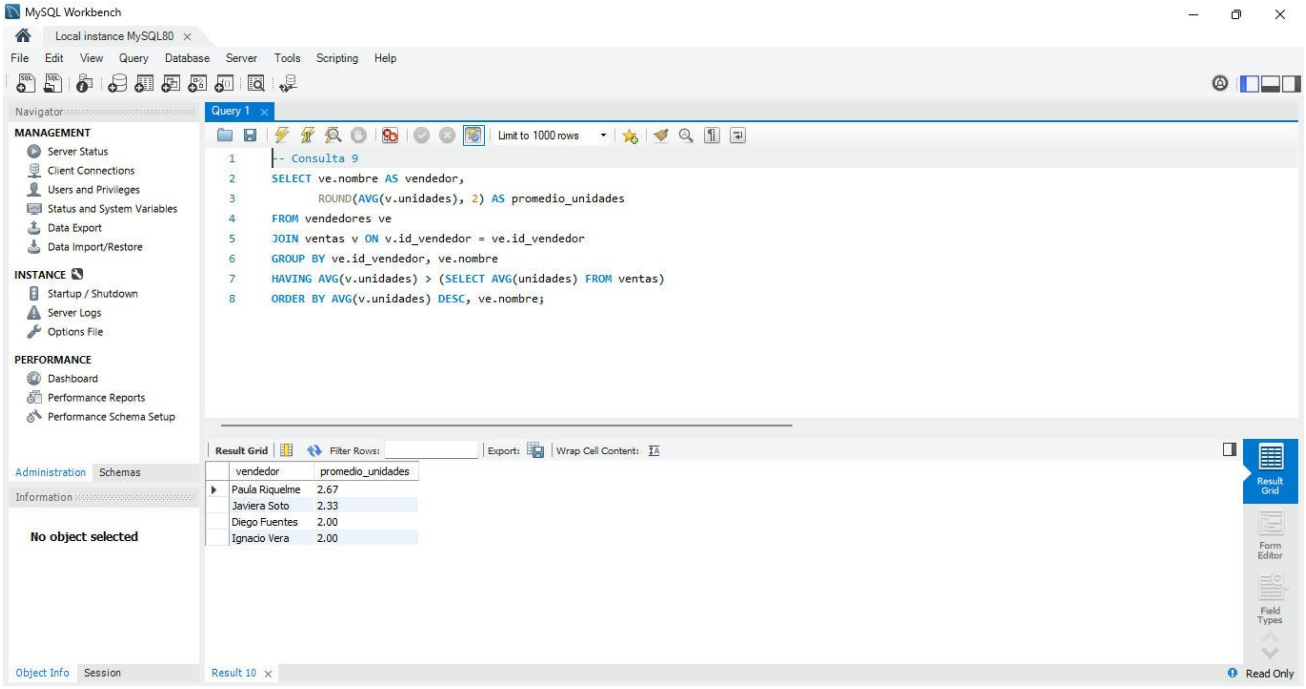
```
SELECT modelo, precio
FROM bicicletas
WHERE precio > (SELECT AVG(precio) FROM bicicletas)
ORDER BY precio DESC;
```

El promedio del catálogo es $657.490; se obtuvo con la subconsulta SELECT AVG(precio) FROM bicicletas, que promedia el precio de los 8 modelos sin considerar las ventas.



Consulta 9

```
SELECT ve.nombre AS vendedor,
       ROUND(AVG(v.unidades), 2) AS promedio_unidades
FROM vendedores ve
JOIN ventas v ON v.id_vendedor = ve.id_vendedor
GROUP BY ve.id_vendedor, ve.nombre
HAVING AVG(v.unidades) > (SELECT AVG(unidades) FROM ventas)
ORDER BY AVG(v.unidades) DESC, ve.nombre;
```



Consulta 10

```
SELECT ve.nombre AS vendedor,
       SUM(v.unidades * b.precio) AS monto_total
FROM vendedores ve
LEFT JOIN ventas v     ON v.id_vendedor  = ve.id_vendedor
LEFT JOIN bicicletas b ON b.id_bicicleta = v.id_bicicleta
GROUP BY ve.id_vendedor, ve.nombre
ORDER BY monto_total DESC;
```

Fernanda Lillo y Rubén Castro muestran NULL en vez de cero porque no tienen ventas y SUM sobre filas sin valores devuelve NULL; para que mostraran 0 habría que usar COALESCE(SUM(v.unidades * b.precio), 0).

